

Reviewer Pack — Conway Temperature Bounds Tree-Resolution Width for 3-SAT

Entry 18df83a46e71 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 06:17:36 UTC

Conway Temperature Bounds Tree-Resolution Width for 3-SAT

Entry ID: 18df83a46e71 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 06:17:36 UTC

1. Conjecture

Field A (mathematical branch): Conway-Berlekamp combinatorial game theory: left/right stops and temperature of finite alternating scoring games (Conway 1976; Berlekamp-Wolfe 1994; Siegel 2013). This corner of CGT has essentially no presence in proof complexity outside Beck’s positional-game tradition; a focused arXiv search for ‘Conway temperature’ AND (‘resolution’ OR ‘CNF’ OR ‘proof complexity’) returns no hits. **Field B** (complexity object): Tree-like Resolution refutation width $w(F)$ and size $s(F)$ for unsatisfiable 3-CNFs, viewed in the Cook-Reckhow-Krajicek bounded-arithmetic framework where $\neg F$ is a Σ^b_0 statement whose $V^{0/S}_1$ witnessing complexity is tied by Ben-Sasson-Wigderson to Resolution width and by Pudlak-Impagliazzo to the Prover-Adversary game.

Statement:

For unsat 3-CNF F on $n \leq 14$ variables, define the alternating integer-scoring game $G(F)$: players A (Maximizer) and B (Minimizer) alternate; on each turn the active player picks one unassigned variable and freely assigns a value; when all variables are assigned the score equals the number of falsified clauses. Compute the Conway left stop $L(F)$ (Maximizer to move) and right stop $R(F)$ (Minimizer to move) by memoized minimax on the full 2^n game tree, and let $t(F) := (L(F) - R(F))/2$ be the (degree-0) Conway temperature. Conjecture: (i) $t(F) \leq C_1 \cdot w(F)$ for every unsat 3-CNF F , where $w(F)$ is the tree-Resolution refutation width, and (ii) $t(F) \geq C_2 \cdot n$ for Tseitin formulas $T(G, \omega)$ on 3-regular expander graphs G with $v \leq 10$ vertices; a single instance violating either bound refutes the conjecture (defaults $C_1 = 5$, $C_2 = 0.3$).

Rationale (proposer’s reasoning):

Tree-Resolution width is controlled by adversarial variable choice in the Prover-Adversary game, while the Conway temperature measures opposite-sign oscillation in optimal play of a scoring game — a structurally distinct invariant never (to our knowledge) computed on SAT-derived games. Tseitin formulas on expanders have $\Omega(n)$ resolution width by parity-flow, and the parity structure should also force balanced left/right moves at every vertex-clause group, yielding high temperature. If the bridge holds it gives a witnessing-style certificate of width inside the bounded-arithmetic Σ^b_0 Cook-Reckhow framework.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4dfada06c2e95171

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds generating random unsat 3-CNFs ($n \in \{8, 10, 12, 14\}$, $m = \lceil 4.5n \rceil$) and Tseitin formulas on 3-regular graphs ($v \in \{6, 8, 10\}$, odd charge), compute $t(F) = (L(F) - R(F))/2$ by memoized minimax and $w(F)$ by *exact tree-DPLL*. *SUPPORT iff every random instance satisfies $t(F) \leq 5 \cdot w(F)$ AND every Tseitin instance satisfies $t(F) \geq 0.3 \cdot v$; one violation falsifies.*

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Conway temperature combinatorial game theory resolution width proof complexity - alternating minimax game 3-SAT Tseitin expander resolution refutation width - left right stop scoring game tree resolution CNF Prover Adversary

Top relevant hits considered: - [<http://arxiv.org/abs/1709.03479v2>] Conway’s potential function via the Gassner representation - [<http://arxiv.org/abs/2203.14669v1>] Dynamic Structure in Four-strategy Game: Theory and Experiment - [<http://arxiv.org/abs/2207.14140v1>] Playing a 2D Game Indefinitely using NEAT and Reinforcement Learning - [<http://arxiv.org/abs/2501.08313v1>] MiniMax-01: Scaling Foundation Models with Lightning Attention - [<http://arxiv.org/abs/2506.13585v1>] MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention - [<http://arxiv.org/abs/1104.3098v1>] Optimal strategies for a game on amenable semigroups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=4.2s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_3cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
```

```

        clause = random.sample(variables, 3)
        for i in range(3):
            if random.random() < 0.5:
                clause[i] = -clause[i]
        clauses.append(tuple(clause))
    return clauses

def generate_tseitin_formula(v, odd_charge=True):
    variables = list(range(1, v + 1))
    edges = []
    for i in range(v):
        for j in range(i + 1, v):
            if random.random() < 0.5:
                edges.append((variables[i], variables[j]))
    if odd_charge:
        if len(edges) % 2 == 0:
            edges.append((variables[0], variables[1]))
    clauses = []
    for u, v in edges:
        new_var = max(variables) + 1
        variables.append(new_var)
        clauses.append((u, v, -new_var))
        clauses.append((u, -v, new_var))
        clauses.append((-u, v, new_var))
        clauses.append((-u, -v, -new_var))
    return clauses

def is_unsat(clauses):
    n = max(abs(lit) for clause in clauses for lit in clause)
    assignments = {}

    def backtrack():
        if len(assignments) == n:
            return not any(all(assignments.get(abs(lit), False) == (lit < 0) for lit in clause) for clause in clauses)
        var = min(set(range(1, n + 1)) - set(abs(lit) for lit in assignments))
        for val in [True, False]:
            assignments[var] = val
            if backtrack():
                return True
            del assignments[var]
        return False

    return backtrack()

def minimax(clauses, player='max'):
    n = max(abs(lit) for clause in clauses for lit in clause)
    memo = {}

    def evaluate(assignments):
        if len(assignments) == n:
            return sum(1 for clause in clauses if all(assignments.get(abs(lit), False) == (lit < 0) for lit in clause))
        key = (tuple(sorted(assignments.items())), player)
        if key in memo:
            return memo[key]
        var = min(set(range(1, n + 1)) - set(abs(lit) for lit in assignments))
        if player == 'max':

```

```

        best = -float('inf')
        for val in [True, False]:
            new_assignments = assignments.copy()
            new_assignments[var] = val
            best = max(best, evaluate(new_assignments))
        memo[key] = best
        return best
    else:
        best = float('inf')
        for val in [True, False]:
            new_assignments = assignments.copy()
            new_assignments[var] = val
            best = min(best, evaluate(new_assignments))
        memo[key] = best
        return best

    return evaluate({})

def tree_dp11(clauses):
    n = max(abs(lit) for clause in clauses for lit in clause)
    memo = {}

    def solve(assignments):
        if len(assignments) == n:
            return not any(all(assignments.get(abs(lit), False) == (lit < 0) for lit in clause) for clause in clauses)
        key = tuple(sorted(assignments.items()))
        if key in memo:
            return memo[key]
        var = min(set(range(1, n + 1)) - set(abs(lit) for lit in assignments))
        result = False
        for val in [True, False]:
            new_assignments = assignments.copy()
            new_assignments[var] = val
            if solve(new_assignments):
                result = True
                break
        memo[key] = result
        return result

    return solve({})

def run_trial(seed):
    random.seed(seed)
    n_values = [8, 10, 12, 14]
    v_values = [6, 8, 10]
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        m = int(math.ceil(4.5 * n))
        clauses = generate_random_3cnf(n, m)
        if not is_unsat(clauses):
            continue
        instances_tested += 1
        L = minimax(clauses, 'max')

```

```

R = minimax(clauses, 'min')
t = (L - R) / 2
w_star = tree_dp11(clauses)
if t > 5 * w_star:
    conjecture_holds = False
    counterexample = f"Random 3-CNF with n={n} violated  $t(F) \leq 5 \cdot w^*(F)$ : t={t}, w*={w_star}"
    break

if conjecture_holds:
    for v in v_values:
        clauses = generate_tseitin_formula(v)
        if not is_unsat(clauses):
            continue
        instances_tested += 1
        L = minimax(clauses, 'max')
        R = minimax(clauses, 'min')
        t = (L - R) / 2
        if t < 0.3 * v:
            conjecture_holds = False
            counterexample = f"Tseitin formula with v={v} violated  $t(F) \geq 0.3 \cdot v$ : t={t}, v={v}"
            break

    return {
        "metric_name": "temperature_to_width_ratio",
        "metric_value": t / w_star if conjecture_holds else 0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    if metric_values:
        mean = sum(metric_values) / len(metric_values)
        std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    else:
        mean = std = 0

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_s
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
: 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.0, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 2, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.5, w*=True'}
CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.5, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.5, w*=True'}
CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.5, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 2, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.5, w*=True'}
CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.5, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 2, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=14 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=9.5, w*=True'}
CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.0, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.0, w*=True'}
CNF with n=10 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.0, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=5.5, w*=True'}
CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=5.5, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.0, w*=True'}
CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=6.0, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=5.5, w*=True'}
CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=5.5, w*=True'}
TRIAL: {'metric_name': 'temperature_to_width_ratio', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'Random 3-CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=5.5, w*=True'}
CNF with n=8 violated  $t(F) \leq 5 \cdot w^*(F)$ : t=5.5, w*=True'}
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

Metric definition bug: every counterexample reports ' $w=True$ ', a boolean, not an integer tree-Resolution width. The conjecture says $t(F) \leq 5 \cdot w(F)$, where $w(F)$ should be an integer ≥ 1 (often $\Omega(n)$ for random 3-CNF near threshold). If True is being coerced to 1, then $5 \cdot w = 5$ and any $t \geq 5.5$ trivially 'falsifies' the bound — but this is an artifact of the width oracle returning a satisfiability flag rather than a width. The reported falsification is therefore not a real counterexample; the test i

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The ' w ' field is reported as the boolean True rather than an integer tree-Resolution width, so the comparison $t(F) \leq 5 \cdot w(F)$ is being evaluated against a coerced value of 1 rather than the actual refutation width. The reported falsifications are artifacts of a broken width oracle, not genuine counterexamples. | next: Fix the tree-Resolution width computation to return an integer (exact tree-DPLL depth/width), re-run the 30-seed protocol, and additionally verify the Tseitin lower-bound clause se

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	304160
2	preregistration	claude_max	opus	0	0	6731
3	novelty	claude_max	opus	0	0	3451
4	novelty	claude_max	opus	0	0	6392
5	test_gen	mistral	codestral-latest	0	0	311892
6	test_gen	mistral	codestral-latest	0	0	315077
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	287953
8	test_gen	mistral	codestral-latest	0	0	311112
9	critic	claude_max	opus	0	0	13158
10	judge	claude_max	opus	0	0	10105

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1570031 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/18df83a46e71.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/18df83a46e71.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/18df83a46e71.tar.gz` (if generated)